Unmatched (PHASE 2)

Bu-Ali Sina University

Mohadese Nejatbakhsh 40412358054

Dina Sharifypanah 40412358025

Dr. Yousef-Sanati

Contents

Introduction

This project is a graphical implementation of the Unmatched board game, developed in C++ using the Raylib library. The game is a turn-based strategy game in which players control heroes and their sidekicks, move them across the board, use cards, and fight against their opponents.

The main goal of the graphics phase was to transform the game logic into an interactive and user-friendly graphical interface. Raylib was used to create the game window, menus, buttons, board, characters, cards, animations, and other visual elements. The graphical interface also provides features such as New Game, Load Game, Save Game, player setup, hero selection, character placement, and combat interaction.

Overall, this phase connects the previously implemented game logic with a visual interface, allowing players to interact with the game through graphical elements instead of the console.


Herald
STRANGE OCCURRENCES

Invisible Man

Invisible Man is a melee hero with 15 health and 2 movement points. His special ability allows him to move directly between spaces containing Fog tokens. Additionally, when defending while standing on a Fog, he gains +1 defense value.

Fog is a special token placed on the board that interacts with Invisible Man's abilities. It allows him to use his special movement ability and provides him with an additional defensive advantage when he is standing on it.

Save and Load

The Save and Load system allows players to save their current game progress and continue it later. The Save feature stores important game information, including players, heroes, sidekicks, their positions, cards, and the current turn. The Load feature reads the saved data and restores the game state, allowing the player to continue the game from where it was previously saved.

```
void SaveManager::saveGame(Game *game, const string &saveName)
{
    if (game == nullptr)
    {
        throw runtime_error("Game pointer is null.");
    }

    string path = getSavePath(saveName);

    game->saveGame(path);
}

void SaveManager::loadGame(Game *game, const string &saveName)
{
    if (game == nullptr)
    {
        throw runtime_error("Game pointer is null.");
    }

    if (!saveExists(saveName))
    {
        throw runtime_error("Save file does not exist: " + saveName);
    }

    string path = getSavePath(saveName);
    game->loadGame(path);
}
```

saveGame()

Receives the current Game object and a save name, creates the corresponding file path, and calls the game's saveGame() function to store the current game state.

loadGame()

Checks whether the selected save file exists and then loads its data into the current Game object using game->loadGame(). This allows the player to continue a previously saved game.

Asset Manager

The AssetManager class is responsible for loading, storing, providing access to, and releasing the graphical and audio assets used throughout the game. It centralizes resource management so that different parts of the game can access textures, fonts, cards, character images, icons, and music without loading them repeatedly.

load()

Loads all game resources, including backgrounds, fonts, character textures, card images, action icons, fog textures, and background music. It also checks whether essential resources were loaded successfully. If a required resource fails to load, unload() is called and the function returns false.

unload()

Releases all loaded textures, fonts, and other resources from memory. It also clears the character and card texture maps. This prevents unnecessary memory usage and resource leaks.

isTextureValid()

Checks whether a texture was loaded successfully by verifying that its texture ID is not zero.

```
bool AssetManager::isTextureValid(const Texture2D &texture) const
{
    return texture.id != 0;
}

bool AssetManager::load()
{
    mainMenuBackground = LoadTexture("Unmatched_Assets/main_menu.png");
    loadingBackground = LoadTexture("Unmatched_Assets/loading.png");
    gameMap = LoadTexture("Unmatched_Assets/board.png");
    MainPanelBackground = LoadTexture("Unmatched_Assets/mainPanel.png");
    winnerBackground = LoadTexture("Unmatched_Assets/gameOver.png");

    gameFont = LoadFont("Unmatched_Assets/fonts/Sweet Magic.ttf");
    titleFont = LoadFont("Unmatched_Assets/fonts/Lost Saloon.ttf");
    loading = LoadFont("Unmatched_Assets/fonts/NexaRustSlab.ttf");
    guideFont = LoadFont("Unmatched_Assets/fonts/guide.TTF");

    gameMusic = LoadMusicStream("Unmatched_Assets/sounds/MainMusic.mp3");
    SetMusicVolume(gameMusic, 0.6f);
    gameMusic.looping = true;

    characterTextures["dracula"] = LoadTexture("Unmatched_Assets/dracula/DracArtTran.png");
    characterTextures["dracula_art"] = LoadTexture("Unmatched_Assets/dracula/DracArt.png");
    characterTextures["dracula_art_transparent"] = LoadTexture("Unmatched_Assets/dracula/DracArtTran.png");
```

```
void AssetManager::unload()
{
    if (mainMenuBackground.id != 0)
    {
        UnloadTexture(mainMenuBackground);
        mainMenuBackground = {};
    }

    if (loadingBackground.id != 0)
    {
        UnloadTexture(loadingBackground);
        loadingBackground = {};
    }

    if (gameMap.id != 0)
    {
        UnloadTexture(gameMap);
        gameMap = {};
    }
```

MapCoordinates

The MapCoordinates component defines the graphical position and size of all **32 spaces** on the game board. It stores the screen coordinates of each space so that game objects such as fighters, fog tokens, movement indicators, and other UI elements can be displayed at the correct location on the map.

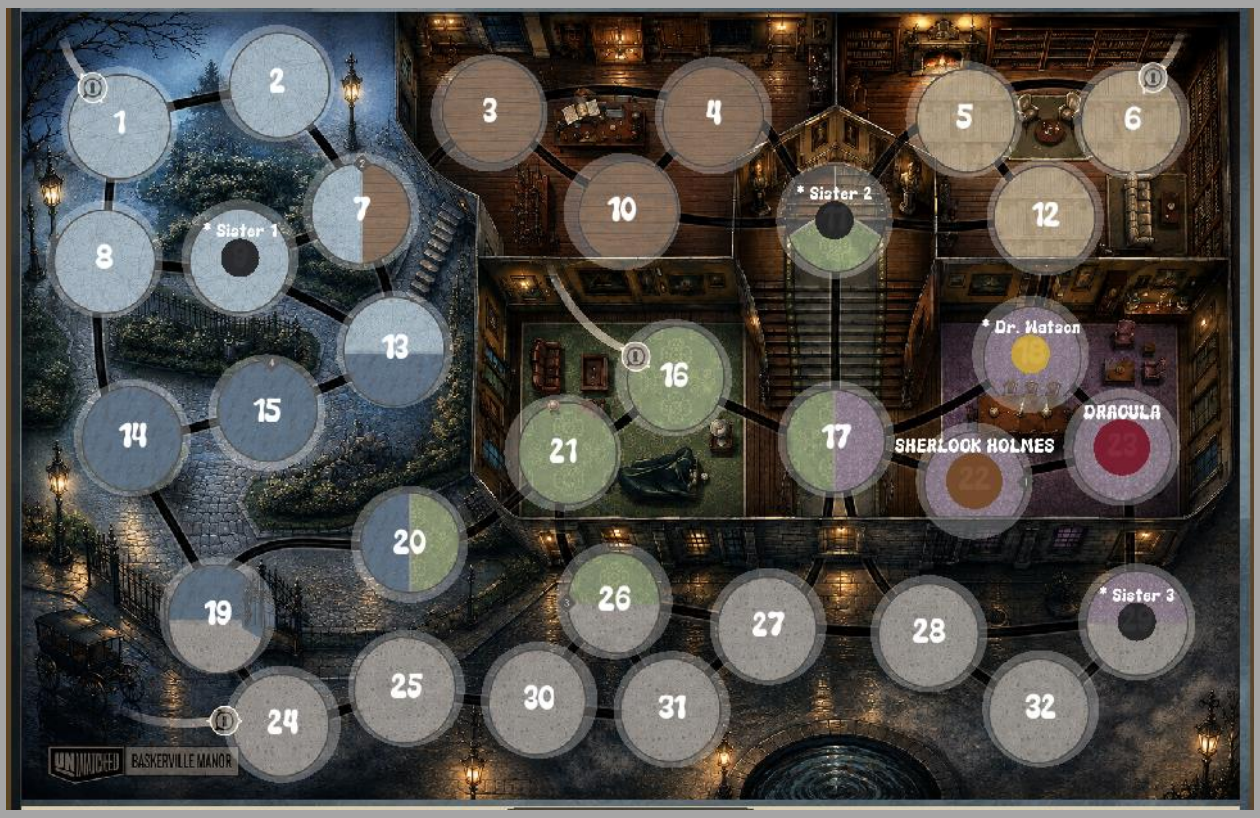
Important Part

SPACE_GRAPHICS[32]

A constant array containing the graphical information for all 32 map spaces. Each element stores:

- The **X and Y coordinates** of the center/position of the space.
- The **radius/size** of the space, which is set to 75.0f.

These predefined coordinates provide a fixed connection between the game's logical map spaces and their visual positions on the Raylib game board. Therefore, when a fighter is located on a specific space, the game can use its corresponding entry in SPACE_GRAPHICS to determine where it should be drawn on the screen.


1
2
3
4
5
6
7
8
* Sister 1
10
* Sister 2
12
13
* Dr. Watson
16
14
15
17
DRACULA
21
SHERLOCK HOLMES
22
20
19
26
27
28
* Sister 3
24
25
30
31
32
UNMATCHED
BASKERVILLE MANOR

Main menu

MainMenu implements the entire pre-game user interface flow. the main menu, player registration, hero selection, and initial board placement. It acts as a finite-state machine driven by a State enum, where each state corresponds to a distinct screen (main menu, player input, load game, hero selection, placement).

MainMenu owns no game logic itself. it reads and writes state through a Game* pointer, delegating actual rule enforcement (turn order, board occupancy, hero assignment) to Game and Board.

Every frame, the game loop calls two methods on this class: update() (handles keyboard input for text fields) and handleInput() (handles mouse clicks and returns an integer signal telling main.cpp what to do next. A separate draw() method renders the current state.

update()

Handles raw keyboard input only for the two player-registration states (PLAYER_1_INPUT, PLAYER_2_INPUT). It reads characters via GetCharPressed() and appends them to

either the name or age string depending on which field (enteringName/enteringAge) is currently focused, filters age input to digits only, supports backspace, and moves focus from name to age when TAB or ENTER is pressed.

draw()

A single large function that branches on state and renders the appropriate screen: title, buttons for New Game/Load Game/Exit, player registration boxes, the save-file list, the "Ready" confirmation screen, and (via helper calls) hero selection and board placement. All UI elements use immediate-mode Raylib drawing (DrawRectangleRounded, DrawTextEx), with hover-based color changes computed by checking CheckCollisionPointRec against the current mouse position each frame.

handleInput()

This is the most important function. It's called once per frame and processes mouse clicks appropriate to the current state:

- **MAIN_MENU**: detects clicks on New Game / Load Game / Exit, resets registration fields, and transitions state

accordingly. Returns a numeric code (1, 2, 3) that main.cpp uses to trigger side effects.

- **PLAYER_1_INPUT / PLAYER_2_INPUT**: on clicking "Next"/"Finish", validates that both name and age were entered (isPlayer1Complete()/isPlayer2Complete()) before advancing.

- **LOAD_GAME**: on clicking a save-slot button, delegates to SaveManager::loadGame(game, filename) to restore game state from disk, returning 5 to signal "resume into GameScreen without running the normal setup pipeline."

- **READY**: on clicking Start, calls startHeroSelection().

- **HERO_SELECTION**: detects hero box clicks, calls selectHero(...), and once both players have chosen, calls game->startGame() (builds decks, draws initial hands) followed by startPlacement().

- **PLACEMENT**: delegates entirely to updatePlacement(); once placement is FINISHED, returns 4, the signal that tells main.cpp to fade transition into the actual game screen.

Hero Selection Logic

- startHeroSelection():
  calls game->initialize(age1, age2) to create the two Player objects and determine who is younger (by age, or randomly if tied). Sets currentHeroPlayer so the younger player picks first this ordering is a core game rule.

- selectHero(const std::string &hero):
  asks game->assignHero(playerIndex, hero) to actually construct the Hero object and its sidekicks/fog tokens On success, flips currentHeroPlayer so control passes to the other player.

- drawHeroSelection(): renders the three hero choice boxes with hover/selected highlighting and enables a "Finish" button only once bothHeroesSelected() is true.

Placement Logic

Placement follows a strict sequence, tracked by the Placement enum: YOUNGER_HERO, YOUNGER_SIDEKICKS, OLDER_HERO, OLDER_SIDEKICKS, FOG, FINISHED (the sidekick and fog stages are skipped automatically if a hero has none).

- **placeHeroOnSpace(int spaceId)**: validates the space is unoccupied, moves the hero there via Board::moveFighter, and decides the next placement phase based on whether the player has sidekicks and whether fog tokens exist anywhere in the game.

- **isValidSidekickPlacement(Space*)**: enforces the rule that sidekicks must be placed in the same zone as the hero's starting space (Board::sameZone).

- **placeSidekickOnSpace(int spaceId)** / **placeFogOnSpace(int spaceId)**: place one sidekick/fog token at a time, advancing an index (placementSidekickIndex, currentFogIndex) and calling

finishPlacement() once all of a player's sidekicks/fogs are placed, which computes the next Placement phase.

- **updatePlacement()**: on each mouse click, converts screen coordinates to board-space coordinates (via mapImageToScreen/getMapScale) and checks which of the 32 map spaces was clicked, dispatching to the appropriate placement function based on the current Placement phase.

- **drawPlacement()**: renders the board with color-coded highlight circles. valid spaces (yellow/blue tint depending on phase), the space under the mouse cursor (white), and already-occupied spaces (purple for hero) giving the player real-time visual feedback about where they're allowed to click.

AttackUI

Handles the graphical flow for a player initiating an attack: picking the attacking fighter, an attack card, a target, and (for the opponent) a defense response.

openAttack(Player*, vector<Fighter*>, Game*)

Resets all UI selection state and builds the list of alive, selectable fighters for the current player, then switches the UI into the SelectAttacker phase.

update()

Main per-frame input loop; advances the UI through its phases (attacker → card → target → confirm) based on mouse clicks.

draw()

Renders the current phase's buttons/cards/highlights to the screen.

ManeuverUI

Handles the graphical flow for a player's "maneuver" action: asking whether to move, choosing which fighter moves, optionally burning a card for extra movement, and picking a destination space.

update()

Per-frame input handling that walks through the phases (ask → select fighter → optionally burn a card → select destination space).

beginSpaceSelection(vector<Space*>&moves) / needsAvailableMoves() / getAvailableMoves()

Manage the phase where the game waits for the player to click a valid destination among the computed legal moves.

burnSelectedCard()

Applies the effect of discarding a card to increase the movement budget.

SchemeUI

Handles the graphical flow for playing a "Scheme" card from the player's hand.

openScheme(const Hand&hand)

Scans the player's hand for Scheme-type cards (up to 7), and opens the UI; if none exist, shows an "empty hand" message with just a back button instead.

update()

Per-frame input handling for selecting a card or confirming the choice.

draw()

Renders the selectable scheme cards and confirm/back buttons.

EffectUI

Handles the graphical flow for any card effect that requires player input (choosing a space, a fighter, a card, a fog token, a yes/no option, etc.). It acts as a dispatcher: based on the effect's EffectInputKind, it sets up the right selection UI, collects the player's clicks, and packages the result into an EffectChoice for the effect to consume.

```
enum class EffectInputKind
{
    None,
    ChooseAdjacentEmptySpace,
    ChooseReachableSpace,
    ChooseOpponentCardToBurn,
    ChooseEnemyFighter,
    ChooseTwoCardsAndOrder,
    ChooseFighterMoveThenFogMove,
    ChooseLurkingOption,
    ChooseFogSourceAndDestination,
    ChooseFogAndDestination,
    ChooseEnemyAndFogDestination,
    ChooseDefeatedSisterAndZoneSpace,
    ChooseCardsToDiscard,
    ChooseAnyEmptySpace,
    ChooseFighterAndReachableSpace,
    ChooseTargetAdjacentEmptySpace,
    ShowOpponentHand
};
```

EffectInputKind(in Effect.h): a list of all the possible "types of input an effect needs from the player" (e.g. choosing an empty space, choosing an enemy fighter, choosing cards to discard, or choosing a fog and its destination). EffectUI uses this enum to decide which selection UI to display.

open(Game*, Effect*, Fighter*, Fighter*)

Entry point; resets all selection state, reads the effect's required EffectInputKind, and dispatches to the matching setupChoose...() function to build that specific selection UI.

setupChooseAdjacentEmptySpace() / setupChooseReachableSpace() / setupChooseAnyEmptySpace()

Prepare board-space selection UIs with different range/validity rules (adjacent only, full reachable range, or any empty space).

setupChooseFogSourceAndDestination() / setupChooseFogAndDestination() / setupChooseEnemyAndFogDestination()

Handle the more complex, multi-step Fog-related effects (pick a fog token, then pick where it moves).

GameScreen

The GameScreen class is responsible for displaying and managing the main game interface. It connects the game logic with the graphical elements, including the board, fighters, cards, player information, buttons, and combat actions. It also handles user interactions such as selecting cards, moving fighters, attacking, and saving the game.

update()

The core per-frame logic; checks mouse input and game state each frame, and delegates to whichever sub-UI is currently open (hand view, Dracula's special ability, feint-blocked popup, etc.) in priority order, then handles normal turn actions when nothing else has focus.

draw()

Top-level rendering call; draws the map, fighters, fogs, panels, buttons, and any open popups for the current frame.

getClickedSpaceId()

Converts the current mouse position into the board-space ID the player clicked on, accounting for the map's zoom/offset transform.

refreshSaveFiles() / updateSaveMenu() / drawSaveMenu()

Handle the in-game save/load menu (listing save files, saving, or loading mid-game).

consumeGameOver(Hero*&outWinner)

One-shot check for whether the game has ended, returning the winning hero if so, so the caller (main loop) can switch to an end-game screen.

```
enum class AttackPhase
{
    NONE,
    CHOOSE_ATTACKER,
    CHOOSE_TARGET,
    CHOOSE_ATTACK_CARD,
    CHOOSE_DEFENSE,
    RESOLVE
};
```

The AttackPhase enum represents the different stages of an attack in the game. It is used to control the flow of the combat interaction and determine what the player should do at each step.

Save Manager

The SaveManager class is responsible for managing the game's save files. It handles creating save files, checking their existence, loading previously saved games, and deleting saves. It also manages the save directory using the C++ filesystem library.

saveGame()

Creates the correct file path for the selected save and stores the current game state by calling the game's saveGame() function.

loadGame()

Checks whether the selected save exists and then loads its data into the current Game object, allowing the player to continue the saved game.

The Game::saveGame() and Game::loadGame() functions perform the main saving and loading operations, while SaveManager manages the save files and provides file-related operations such as creating, finding, and deleting saves.

Transition

The Transition class is responsible for creating smooth visual transitions between different screens of the game. In this project, it is mainly used for fade-to-black effects when switching between the Main Menu and the Game Screen. It can also display a message in the center of the screen during the transition.

start()

Starts a new transition and sets its type, duration, and black-screen hold duration.

update()

Updates the transition timer according to the frame's deltaTime and determines when the transition is finished.

draw()

Draws the fade effect using a semi-transparent black overlay. During the black-screen part, it also displays the transition text.

shouldSwitch()

Determines whether the screen should switch to the next screen, usually when the fade reaches the black stage.

Player Panel

The PlayerPanel class is responsible for displaying each player's information during the game. It provides a visual representation of the player's name, hero, health, cards, and other game-related information. It also helps make the game interface more interactive and easier to understand.

Dracula Ability UI

The DraculaAbilityUI class provides the graphical interface for Dracula's special ability. It allows the player to choose whether to activate the ability, select a valid adjacent fighter as the target, and then apply the ability's effect. The interface is implemented using Raylib and includes buttons, fighter images, health information, and selection states.

applyAbility()

Applies Dracula's ability to the selected fighter by dealing 1 damage and allowing Dracula's player to draw a card.

WinnerUI

The WinnerUI class is responsible for displaying the winner screen after the game ends. It shows the winning hero, a background, a victory message, and an option to return to the Main Menu.

Challenges

One of the main challenges of this project was implementing the graphical interface using Raylib. Developing and coordinating different screens, buttons, animations, and game elements was more difficult and time-consuming than expected. In the early stages, we also faced problems caused by differences between our graphics cards and system configurations. These issues affected the way the game window and graphical elements were displayed, and we spent several days troubleshooting and fixing them.

Another challenging part was defining the exact coordinates of the board spaces. Since the board had many individual spaces, we had to calculate and adjust the position of each one separately to make sure the fighters and other elements were displayed correctly. This process was very time-consuming and required a lot of testing and adjustment.

Overall, integrating the graphical interface with the existing game logic, handling user interactions, implementing different game states, and making everything work

consistently made the graphics phase a challenging but valuable part of the project.

Assets

The required game assets, including images, fonts, and other graphical resources, can be downloaded from the following Google Drive link:
https://drive.google.com/file/d/1rFYiy6056YsT2NlpfybQbjv8QEl7wZyQ/view?usp=drivesdk

Repository link

The source code, project files, and documentation are available in the following GitHub repository:

https://github.com/momoshinz/Unmatched-Graphics.git

Resources

- Pixlr E website
- ChatGPT ai
- Raylib
- YouTube